Tecnología para la Gestión II

Estadística descriptiva

De la teoría a `group_by()` y `summarise()`

Francisco Villarroel Riquelme | 2026-10-07

Administración Pública · Escuela de Gobierno · Facultad de Economía y Gobierno · ADPU B012

¿Qué veremos hoy?

Bloque 1 — Conceptos

- Estadística descriptiva vs. inferencial
- Frecuencias y porcentajes
- Medidas de tendencia central
- Medidas de dispersión y posición

Bloque 2 — En R

- Cada concepto, con su función en R
- `group_by()` y `summarise()`
- Estadísticas por grupo con la base municipal
- Ejercicios

Sin fórmulas matemáticas: nos enfocamos en qué es cada estadístico, para qué sirve y cuándo conviene usarlo.

Bloque 1

Estadística descriptiva: los conceptos

Estadística descriptiva ≠ estadística inferencial

Descriptiva

Resume y describe las características básicas de los datos que tienes. Te da una comprensión inicial de patrones y tendencias.

Inferencial

Usa una muestra para deducir características de la población completa: predecir, comparar grupos, evaluar hipótesis.

Hoy nos quedamos en la **descriptiva**: resumir lo que ya tenemos en la base de datos. La visualización de datos (que vimos con `ggplot2`) también es una herramienta descriptiva.

Frecuencias y porcentajes

Frecuencia

Conteo de cuántas observaciones caen en cada categoría de una variable. Es lo primero que se revisa en una variable cualitativa.

Sirve para: detectar errores de datos (categorías vacías o con muy pocos casos son sospechosas).

Porcentaje

La frecuencia expresada en relación al total. Permite comparar grupos de distinto tamaño (ej: comunas con distinta cantidad de solicitudes).

Sirve para: dar contexto — "36 solicitudes" dice poco; "36% de las solicitudes" dice mucho más.

Ambas son la base de cualquier análisis: **antes de calcular cualquier otra cosa, mira cuántos casos hay de cada categoría.**

Frecuencias y porcentajes en R

```r
# Frecuencia: conteo por categoría
datos %>%
  count(tipo_solicitud)

# Porcentaje: la frecuencia sobre el total
datos %>%
  count(tipo_solicitud) %>%
  mutate(porcentaje = round(n / sum(n) * 100, 1))
```

`count()` es un atajo de dplyr: agrupa por la variable indicada y cuenta cuántas filas hay en cada grupo. Es, en el fondo, un `group_by()` + `summarise()` ya armado — lo que veremos en el Bloque 2.

Medidas de tendencia central

Media, mediana y moda

Media

El promedio. El estadístico más usado para variables cuantitativas.

✔ Fácil de calcular e interpretar

✘ Muy sensible a valores extremos

Mediana

El valor que divide la muestra exactamente a la mitad, ordenando los datos de menor a mayor.

✔ Robusta ante valores extremos

✘ Sensible a cambios en la cantidad de datos

Moda

El valor o categoría que más se repite. Es la "más popular".

✔ Sirve también para variables cualitativas

✘ Poco informativa por sí sola

¿Cuándo usar cuál? Si la variable tiene valores extremos (outliers) o está muy sesgada, la **mediana** es más representativa que la media. La **moda** es la única opción para variables cualitativas.

Media, mediana y moda en R

```r
mean(datos$monto_asignado, na.rm = TRUE)      # media
median(datos$monto_asignado, na.rm = TRUE)    # mediana

# la moda no tiene función propia en R base:
# se obtiene viendo qué categoría tiene mayor frecuencia
datos %>%
  count(tipo_solicitud) %>%
  slice_max(n, n = 1)
```

> `na.rm = TRUE` le dice a R que ignore los valores faltantes (`NA`) al calcular. Sin esto, si hay un solo `NA` en la columna, el resultado completo será `NA`.

Medidas de dispersión y posición

¿Qué son las medidas de dispersión?

Las medidas de tendencia central nos dicen **dónde** se agrupan los datos. Las medidas de dispersión nos dicen **qué tan separados** están unos de otros.

Dos grupos pueden tener la misma media y ser completamente distintos: uno muy parejo, otro con casos muy dispersos. La dispersión es lo que marca esa diferencia.

Rango y desviación estándar

Rango

La distancia entre el valor máximo y el mínimo de la variable.

✔ Muy fácil de calcular

✘ Solo mira los extremos: no dice nada de cómo se distribuye el resto de los datos

Desviación estándar

Qué tan lejos están, en promedio, los valores respecto a la media.

✔ Usa toda la información disponible; es la base de muchos análisis posteriores

✘ También se ve inflada por valores extremos, porque se calcula a partir de la media

Regla práctica: media y desviación estándar juntas dan una buena idea de dónde se concentra la mayoría de los casos.

Rango y desviación estándar en R

```r
range(datos$monto_asignado, na.rm = TRUE)    # valor mínimo y máximo
sd(datos$monto_asignado, na.rm = TRUE)       # desviación estándar
```

`range()` devuelve dos números: el mínimo y el máximo. Si quieres solo la distancia entre ellos, puedes restarlos: `diff(range(datos$monto_asignado, na.rm = TRUE))`.

Cuartiles: dividir la distribución en partes

¿Qué son?

Puntos que dividen el conjunto de datos ordenado en partes iguales. Los más usados son los **cuartiles** (4 partes: 25%, 50%, 75%).

✔ Más robustos que la media/desviación estándar ante valores extremos

✔ Muestran la forma completa de la distribución, no solo un resumen

¿Para qué sirven?

Por ejemplo: "el 25% de las solicitudes tarda menos de X días" es más informativo que solo el promedio de días de tramitación.

✘ No se resumen en un único número — hay que mirar varios valores a la vez

Cuartiles en R

```
quantile(datos$dias_tramitacion,
         probs = c(0.25, 0.5, 0.75),
         na.rm = TRUE)
```

> El cuartil 50% (`0.5`) es exactamente la **mediana**. `quantile()` te permite pedir cualquier punto de corte, no solo cuartiles: por ejemplo `probs = 0.9` te da el percentil 90.

Bloque 2

`group_by()` y `summarise()`

El problema: resúmenes "a mano" no escalan

Hasta ahora calculamos estadísticos sobre **toda la base**: `mean(datos$monto_asignado)`. Pero casi siempre queremos saber esto **por grupo**: ¿cuál es el monto promedio en cada comuna? ¿en cada tipo de solicitud?

Filtrar comuna por comuna y calcular la media de cada una, una por una, funciona — pero es lento y propenso a errores. `group_by()` + `summarise()` lo hacen en un solo paso.

`group_by()` — agrupar antes de calcular

¿Qué hace?

Le dice a R: "de aquí en adelante, trata cada categoría de esta variable como un grupo separado". No cambia los datos visualmente, pero marca la base como agrupada.

`summarise()`

Calcula un resumen (media, mediana, conteo, etc.) **para cada grupo** definido por `group_by()`, en vez de para toda la base.

> Van siempre juntos: `group_by()` prepara los grupos, `summarise()` calcula el resumen dentro de cada uno.

La sintaxis

```
datos %>%
  group_by(comuna) %>%
  summarise(promedio_monto = mean(monto_asignado, na.rm = TRUE))
```

> Se lee: *"a `datos`, agrúpalo por `comuna`, y para cada comuna calcula el promedio de `monto_asignado`"*. El resultado es una tabla nueva, con una fila por comuna.

`summarise()` siempre reduce los datos: si tenías 720 filas y agrupas por comuna, el resultado tiene tantas filas como comunas distintas haya.

Varios estadísticos a la vez

```
datos %>%
  group_by(comuna) %>%
  summarise(
    n_solicitudes  = n(),
    promedio_monto = mean(monto_asignado, na.rm = TRUE),
    mediana_dias   = median(dias_tramitacion, na.rm = TRUE),
    monto_maximo   = max(monto_asignado, na.rm = TRUE)
  )
```

`n()` cuenta cuántas filas hay en cada grupo — es la forma de obtener la **frecuencia** dentro de un `summarise()`. Puedes combinar tantos estadísticos como necesites, separados por comas.

Agrupar por más de una variable

```
datos %>%
  group_by(comuna, tipo_solicitud) %>%
  summarise(
    n_solicitudes  = n(),
    promedio_monto = mean(monto_asignado, na.rm = TRUE)
  )
```

Con dos variables en `group_by()`, R crea un grupo por **cada combinación** de comuna y tipo de solicitud. Útil cuando quieres cruzar dos categorías a la vez.

`group_by()` + `summarise()` + el resto de lo que ya sabes

```
datos %>%
  filter(estado != 4) %>%
  group_by(oficina_municipal) %>%
  summarise(
    n_solicitudes    = n(),
    promedio_monto   = mean(monto_asignado, na.rm = TRUE),
    satisfaccion_prom = mean(puntaje_satisfaccion, na.rm = TRUE)
  ) %>%
  arrange(desc(n_solicitudes))
```

Todo el flujo junto: **filtrar** primero (solo solicitudes no rechazadas), **agrupar** por oficina, **resumir** con varios estadísticos, y **ordenar** el resultado con `arrange()` de mayor a menor.

`arrange(desc(...))` no es obligatorio, pero ayuda a leer el resultado: las oficinas con más solicitudes aparecen primero.

Errores frecuentes

De sintaxis

- Usar `group_by()` sin `summarise()` después (no pasa nada visible, pero no calculaste nada)
- Olvidar `na.rm = TRUE` y obtener `NA` en el resultado
- Poner el nombre del estadístico entre comillas dentro de `summarise()`

De lógica

- Agrupar por una variable que tiene casi un valor distinto por fila (el resumen no resume nada)
- Confundir `n()` (cuenta filas) con `sum()` (suma una columna numérica)

Manos a la obra

Ejercicios con la base municipal

Ejercicios

1. Sin agrupar, calcula la media, mediana y desviación estándar de `monto_asignado`.

2. Con `count()`, obtén la frecuencia y el porcentaje de solicitudes por `nivel_prioridad`.

3. Con `group_by()` y `summarise()`, calcula el número de solicitudes y el promedio de `dias_tramitacion` por `comuna`.

4. Agrupa por `tipo_solicitante` y calcula el promedio de `puntaje_satisfaccion` y el monto máximo asignado. Ordena el resultado de mayor a menor promedio de satisfacción.

¡Buen trabajo!

La próxima clase: gráficos con ggplot2

Dudas: fvillarroelr1@docente.uss.cl · Tecnología para la Gestión II · ADPU B012